



KIẾN TRÚC VÀ THIẾT KẾ PHẦN MỀM

ThS. Nguyễn Văn Hữu Hoàng

PGĐ Trung tâm Kinh doanh Công nghệ số, Công ty Dịch vụ MobiFone Khu vực 8

THÔNG TIN GIẢNG VIÊN

- ThS. Nguyễn Văn Hữu Hoàng
- Phó giám đốc Trung tâm Kinh doanh Công nghệ số, Công ty Dịch vụ MobiFone Khu vực 8
- SĐT: 0908.862.626, Email: huuhoangvn@gmail.com
- Chuyên môn: Kiến trúc sư giải pháp, Kiến trúc sư Cloud.
- Kinh nghiệm 18 năm phát triển, triển khai, quản lý các dự án CNTT, các dự án chuyển đổi số:
 - 4 năm trong lĩnh vực tài chính, chứng khoán.
 - 14 năm trong lĩnh vực viễn thông.
- Chứng chỉ quốc tế:
 - Microsoft Certified Professional Developer
 - Microsoft Certified IT Professional
 - Oracle Administrator Certified Professional
 - Google Professional Cloud Architect
 - Google Professional Machine Learning Engineer ...

<https://www.linkedin.com/in/nguyen-van-huu-hoang/>



I MỤC TIÊU CỦA MÔN HỌC

- **Về kiến thức:**

- Mục đích tập trung thiết kế dựa vào thành phần và mẫu thiết kế.
- Hiểu được khái niệm kiến trúc, thành phần và mẫu thiết kế
- Các công nghệ thành phần và mẫu thiết kế
- Nâng cao trình độ của sinh viên về phân tích và thiết kế phần mềm với thành phần và mẫu thiết kế cùng sử dụng công cụ để thiết kế với UML

- **Kỹ năng:**

- Nâng cao kỹ năng phân tích và thiết kế hệ thống
- Sử dụng được các công cụ và công nghệ để vận dụng vào phát triển hệ phần mềm dựa trên thành phần và mẫu thiết kế
- Nâng cao khả năng làm việc nhóm (cộng tác, thảo luận...) giữa các thành viên khác nhau
- Đọc hiểu tài liệu chuyên ngành thiết kế phần mềm bằng tiếng Anh

- **Thái độ, Chuyên cần:**

- Đi học đầy đủ các buổi là yêu cầu quan trọng.

| HỌC LIỆU

Học liệu bắt buộc

- [1] Trần Đình Quế, Giáo trình Kiến trúc và thiết kế phần mềm, 2017
- [2] Andy Wang, Component-Oriented Programming, Wiley Publisher, 2005
- [3] Partha Kuchana, Software architecture design patterns in Java, AUERBACH PUBLICATIONS, 2004

Học liệu tham khảo

- [4] Mike O'Docherty, Object-Oriented Analysis and Design: Understanding System Development with UML 2.0. Publisher John Wiley & Sons, 2005

| TÀI LIỆU MÔN HỌC



<https://bit.ly/bg-ktpm>

| CÔNG CỤ THAM KHẢO

- IDE:

- Java: Netbeans hoặc Eclipse.
- .Net: Visual Studio Code hoặc Visual Studio.

- Thiết kế UML:

- DrawIO (<https://www.draw.io>)
- StarUML (<https://staruml.io>)
- Visual Paradigm
- Rational Rose

I LỊCH TRÌNH CHUNG

Nội dung	Hình thức tổ chức dạy môn học					Tổng cộng
	Lên lớp			Thực hành	Tự học	
	Lý thuyết	BT-TL	Kiểm tra			
Nội dung 1: Chương 1	2					2
Nội dung 2: Chương 2: Biểu diễn kiến trúc và thành phần	4					4
Nội dung 3: Chương 3 Công nghệ thành phần với JavaBeans và EJB	6					6
Nội dung 4: Bài tập Chương 3		2				2
Nội dung 5: Chương 4 Công nghệ thành phần với .NET	6					6
Nội dung 6: Bài tập Chương 5		2				2
Nội dung 7: Kiểm tra giữa kỳ			2			2
Nội dung 8: Chương 5 Mẫu thiết kế cơ bản	4					4
Nội dung 9: Chương 6 Mẫu thiết kế sáng tạo	4					4
Nội dung 10: Bài tập Chương 5&6		2				2
Nội dung 11: Chương 7 Mẫu thiết kế tích hợp	4					4
Nội dung 12: Chương 8 Mẫu thiết kế cấu trúc	2				1	2
Nội dung 13: Sửa bài tập Chương 7&8		2				2
Nội dung 14: Ôn tập và giải đáp môn học	2					2
Tổng cộng	34	8	2		1	45

| HÌNH THỨC KIỂM TRA

Hình thức kiểm tra	Tỷ lệ đánh giá	Đặc điểm đánh giá
- Đi học đầy đủ (trong lớp gây ảnh hưởng đến người khác, mỗi lần nhắc nhở trừ một điểm, mỗi buổi nghỉ học trừ một điểm) - Tích cực thảo luận (không phát biểu buổi nào sẽ được 0 điểm, phát biểu 1 buổi sẽ được 4 điểm)	10 %	Cá nhân
- Trung bình các điểm bài tập lớn	20%	Cá nhân dựa vào bài tập nhóm
- Trung bình các bài kiểm tra trên lớp	10%	Cá nhân
- Kiểm tra cuối kỳ	60%	Cá nhân

| CHƯƠNG 1: KHÁI NIỆM CƠ BẢN VỀ KIẾN TRÚC PHẦN MỀM

- Độ phức tạp phần mềm
- Khái niệm kiến trúc phần mềm
- Một số khái niệm cơ bản khác

| 1.1 TÍNH PHỨC TẠP CỦA PHẦN MỀM

Độ phức tạp của phần mềm là gì?

- *Phần mềm* được hiểu là bao gồm chương trình cùng với dữ liệu và tài liệu liên quan nhằm tự động thực hiện một số chức năng hoặc giải quyết một vấn đề cụ thể nào đó.
- *Độ phức tạp* là bản chất vốn có của một hệ phần mềm. Nó tỷ lệ với kích cỡ của hệ thống, số lượng thành phần cấu thành hệ thống, kích cỡ và cấu trúc bên trong của mỗi thành phần, số lượng và tính chất phụ thuộc nhau giữa các thành phần.
(định nghĩa của Taylor)

I ĐỘ PHỨC TẠP CỦA THÀNH PHẦN PHẦN MỀM

- Một hệ phần mềm với số lượng thành phần lớn hơn sẽ phức tạp hơn so với hệ có số lượng thành phần ít hơn.
- Độ phức tạp của hệ phần mềm còn liên quan đến cả tương tác giữa các thành phần cũng như cấu trúc bên trong và hành vi của riêng các thành phần đó.
- *Môđun hóa* được xem là nguyên tắc cơ bản để đạt được sự đơn giản trong thiết kế phần mềm và có thể làm giảm thiểu cũng như quản lý được độ phức tạp.

I ẢNH HƯỞNG CỦA ĐỘ PHỨC TẠP PHẦN MỀM

- Độ phức tạp nảy sinh qua nhiều giai đoạn phát triển và thể hiện ở nhiều công đoạn trong quá trình phát triển phần mềm.
 - Pha xác định và phân tích yêu cầu
 - Phát thảo giao diện người dùng
 - Thiết kế kiến trúc
 - Thiết kế chi tiết
 - Tạo mã nguồn
- > Tìm cách giảm độ phức tạp của phần mềm.*

I ẢNH HƯỞNG CỦA ĐỘ PHỨC TẠP PHẦN MỀM

- Chúng ta có thể làm giảm độ phức tạp của thiết kế hệ thống bằng cách lựa chọn ngôn ngữ thiết kế hay chọn giao diện phù hợp hoặc cả hai.
- Có thể giảm độ phức tạp bằng cách đưa vào các thuật ngữ chuẩn, như vậy toàn bộ công việc nhằm giảm độ phức tạp phần mềm xoay quanh việc làm sao tạo ra các thuật ngữ để thể hiện các khái niệm và các mối liên quan giữa các khái niệm.
- Đề xuất một số khái niệm như mẫu kiến trúc (architectural pattern), mẫu thiết kế (design pattern), thành phần (component) với hy vọng giảm được độ phức tạp này.

I 1.2 KIẾN TRÚC PHẦN MỀM

- Thuật ngữ **kiến trúc phần mềm** (software architecture) lần đầu tiên được sử dụng vào cuối những năm 1960 nhưng mãi đến những năm 1990 khái niệm này mới được xem xét, nghiên cứu rộng rãi trong công nghệ phần mềm.
- **Kiến trúc hệ thống** (system architecture) là một sự gắn kết các bộ phận trong một hệ thống với nhau bao gồm cấu trúc, giao diện và các cơ chế để phối hợp hoạt động giữa các bộ phận.
- Booch đã định nghĩa **kiến trúc phần mềm** là một loạt các quyết định quan trọng về tổ chức một hệ thống phần mềm: Lựa chọn các thành phần của cấu trúc, giao diện của hệ thống; Xác định kiểu tương tác giữa các thành phần; Xác định các yếu tố cơ bản cấu thành hệ thống; Xác định kiểu kiến trúc của hệ thống; Khả năng tái sử dụng.

I CÁC HƯỚNG DẪN KHI KIẾN TRÚC HỆ THỐNG

- Kiến trúc cần thể hiện được cấu trúc (structure) của hệ thống nhưng ẩn giấu các chi tiết cài đặt.
- Kiến trúc phải hiện thực hóa được tất cả các use case sử dụng.
- Kiến trúc phải thể hiện được sự quan tâm đến các yêu cầu của các bên liên quan.
- Kiến trúc phải đáp ứng cả yêu cầu về chức năng lẫn yêu cầu về phi chức năng.

I 1.3 MỘT SỐ KHÁI NIỆM CƠ BẢN

- Kiến trúc một hệ phần mềm là một tập các quyết định thiết kế chủ yếu được đưa ra cho hệ thống cần xây dựng (Theo Taylor).
- Khái niệm quyết định thiết kế (design decision) là trung tâm của kiến trúc phần mềm nhằm kết nối các khái niệm khác liên quan đến kiến trúc.
- Một số khái niệm liên quan quyết định thiết kế:
 - Các quyết định liên quan đến cấu trúc (structure) của hệ thống. Ví dụ, các thành phần kiến trúc cần phải tổ chức và hợp thành các gói phân cấp theo kiểu 3 tầng.
 - Các quyết định liên quan đến hành vi chức năng (functional behavior). Ví dụ, xử lý dữ liệu, lưu trữ và hiển thị sẽ được thực thi theo một thứ tự nghiêm ngặt.
 - Các quyết định liên quan đến tương tác (interaction). Ví dụ, giao tiếp giữa các thành phần hệ thống chỉ xảy ra bằng cách sử dụng thông báo, sự kiện hay giao thức RMI trong java.
 - Các quyết định liên quan đến các tính chất phi chức năng (nonfunctional properties) của hệ thống. Ví dụ, khả năng phụ thuộc của hệ thống sẽ được đảm bảo bằng cách phục hồi các mô đun xử lý.
 - Các quyết định liên quan đến cài đặt (implementation) hệ thống. Ví dụ, thành phần giao diện người sử dụng sẽ được xây dựng bằng cách sử dụng Java Swing hay JSP.

I MỘT SỐ KHÁI NIỆM CƠ BẢN

- **Thành phần phần mềm** là một thực thể kiến trúc: (i) đóng gói một tập con chức năng và dữ liệu của hệ thống; (ii) chỉ có thể truy nhập vào thực thể thông qua một giao diện; (iii) xác định các phụ thuộc tùy theo ngữ cảnh mà nó yêu cầu.
- **Thành phần phần mềm** là thể hiện đầy đủ các nguyên lý **đóng gói** (encapsulation), **trừu tượng hóa** (abstraction) và **môđun hóa** (modularity) của kỹ thuật phần mềm. Như vậy, các thành phần hàm ý có khả năng kết hợp, sử dụng lại và tiến hóa.
- **Kết nối** là một phần tử kiến trúc có nhiệm vụ thực thi tương tác giữa các thành phần.

I MỘT SỐ KHÁI NIỆM CƠ BẢN

- Các loại kết nối:
 - **Kết nối gọi thủ tục**: ví dụ như phương thức trong lập trình hướng đối tượng.
 - **Kết nối sự kiện**: ví dụ trong ứng dụng windows, các input GUI xem như sự kiện kích hoạt hệ thống.
 - **Kết nối truy nhập dữ liệu**: ví dụ các cơ chế truy vấn trong cơ sở dữ liệu SQL
 - **Kết nối liên kết**: được sử dụng để ràng buộc các thành phần cùng giữ một trạng thái suốt trong quá trình tương tác với nhau.
 - **Kết nối luồng**: được sử dụng để truyền một khối lượng lớn dữ liệu giữa các tiến trình tự chủ riêng. Ví dụ, các socket TCP/UDP hay các giao thức client-server như RMI, CORBA, FTP, SOAP.
 - **Kết nối trung gian**: được sử dụng khi các thành phần biết sự có mặt của các thành phần khác nhưng không thể biết được nhu cầu, trạng thái của nó. Khi đó, bộ phận trung gian sẽ cung cấp dịch vụ phối hợp hay giải quyết tranh chấp...
 - **Kết nối thích nghi**: cung cấp phương tiện để hỗ trợ tương tác giữa các thành phần không được thiết kế với tương tác. Ví dụ, hệ phân tán có thể dựa vào gọi thủ tục từ xa (RPC: remote procedure call) cho mọi tương tác.
 - **Kết nối theo tuyến**: được sử dụng để chỉ ra các tuyến tương tác và thực thi định tuyến truyền thông cũng như phối hợp giữa các thành phần theo tuyến này.

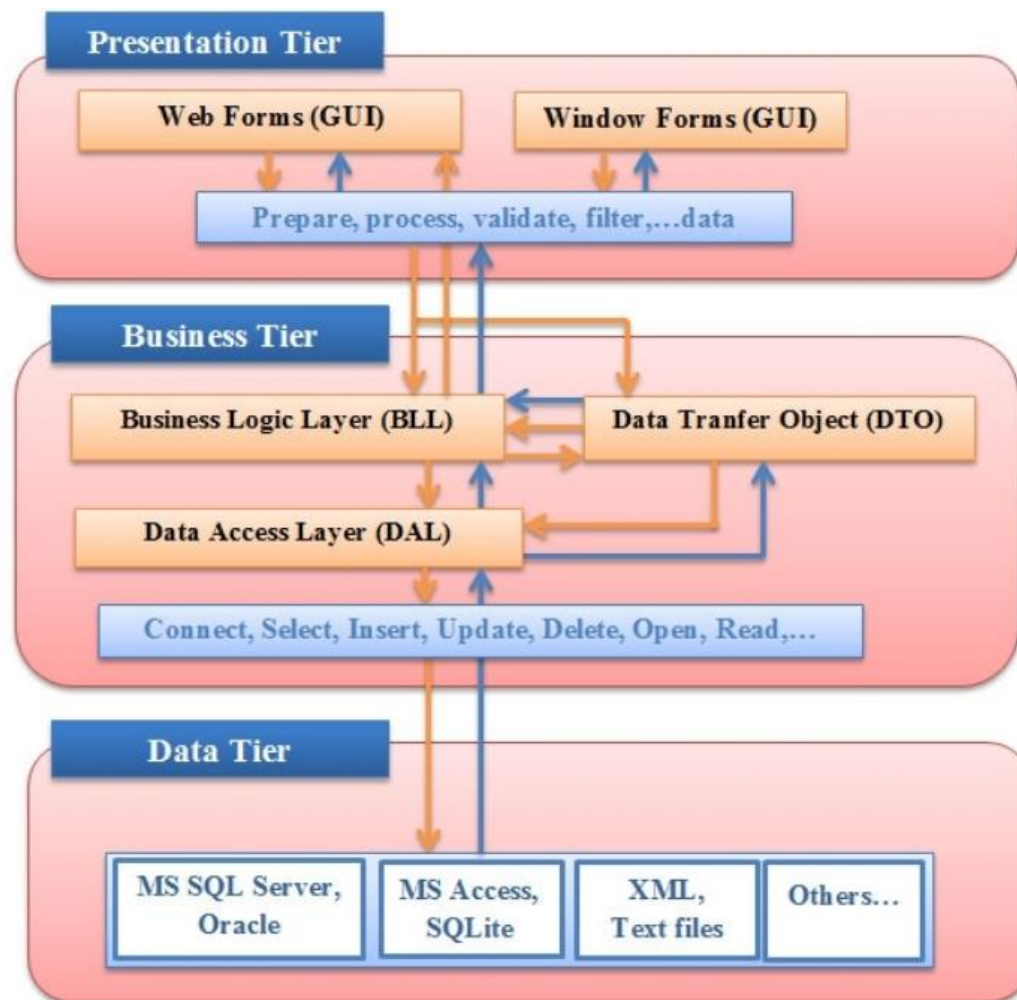
I MỘT SỐ KHÁI NIỆM CƠ BẢN

- **Cấu hình** của kiến trúc là tập các cách kết hợp giữa các thành phần và kết nối của kiến trúc hệ phần mềm. Cấu hình có thể biểu diễn như một đồ thị trong đó các đỉnh biểu diễn thành phần và kết nối, các cạnh biểu diễn sự kết hợp.
- **Kiểu kiến trúc** là tập những quyết định thiết kế: (i) áp dụng được trong ngữ cảnh phát triển; (ii) ràng buộc những quyết định thiết kế cho hệ thống đó; (iii) thể hiện được chất lượng và lợi ích trong kiểu hệ thống như vậy.
- Kiểu kiến trúc thỏa một số điều kiện sau đây sẽ đem nhiều lợi điểm:
 - *Tách biệt về mặt vật lý*: các thành phần phần mềm yêu cầu dịch vụ tách biệt với các thành phần cung cấp dịch vụ. Điều này cho phép cấu trúc các thành phần một cách hợp lý, hiệu quả cả về số lượng yêu cầu và cung cấp dịch vụ.
 - *Trong suốt*: làm cho nhà cung cấp không biết định danh của bên yêu cầu để dễ bề cung cấp dịch vụ một cách trong suốt kể cả khi thay đổi bên yêu cầu.
 - *Cô lập*: tách biệt các bên yêu cầu với nhau để cho phép thêm, loại bỏ hay sửa đổi một cách độc lập.

I MỘT SỐ KHÁI NIỆM CƠ BẢN

- **Mẫu kiến trúc** (architectural pattern) cung cấp một tập các quyết định thiết kế liên quan đến các thành phần và kết nối với những đặc trưng cụ thể cho một lớp hệ phần mềm.
- Phân biệt kiểu kiến trúc và mẫu kiến trúc:
 - **Phạm vi**: Kiểu kiến trúc áp dụng vào một ngữ cảnh phát triển như GUI, trong khi mẫu kiến trúc áp dụng vào một bài toán thiết kế đặc biệt như logic nghiệp vụ của hệ thống phải tách biệt với quản trị dữ liệu.
 - **Trừu tượng hóa**: Kiểu giúp ràng buộc các quyết định thiết kế kiến trúc về hệ thống và khá trừu tượng nên đòi hỏi sự thể hiện của con người. Trong khi đó, mẫu là phần kiến trúc đã được tham số hóa và có thể xem như một mẫu thiết kế cụ thể.
 - **Quan hệ**: Các mẫu không thể sử dụng ngay được mà phải được tham số hóa cho ngữ cảnh của bài toán cụ thể. Nghĩa là một mẫu cũng có thể áp dụng được cho những hệ thống với nhiều kiểu khác nhau. Ngược lại, một hệ thống được thiết kế theo một kiểu nào đó có thể sử dụng nhiều mẫu.

I VÍ DỤ MẪU KIẾN TRÚC 3 LỚP



| Mẫu kiến trúc 3 lớp

- **Tầng client hay trình diễn:** chứa chức năng cần thiết để người dùng truy nhập dịch vụ hệ thống. Tầng này chứa giao diện GUI của hệ thống và có thể lấy dữ liệu và thực thi một số xử lý nhỏ. Tầng client thông thường được triển khai trên các máy với khả năng tính toán và lưu trữ hạn chế như máy để bàn hay các thiết bị cầm tay, di động..
- **Tầng ứng dụng hay tầng nghiệp vụ** hay còn gọi là **tầng trung gian:** chứa chức năng chính của ứng dụng. Tầng này chịu trách nhiệm đáp ứng các yêu cầu dịch vụ hay xử lý yêu cầu từ tầng client và truy nhập, xử lý dữ liệu trong tầng dữ liệu. Tầng này được triển khai trên tập các máy chủ mạnh và lưu trữ lớn.
- **Tầng dữ liệu:** chứa chức năng lưu trữ và truy nhập dữ liệu của ứng dụng. Tầng này chứa cơ sở dữ liệu lớn và có khả năng cung cấp nhiều yêu cầu truy nhập dữ liệu song song.
- Tương tác giữa các tầng về nguyên lý là tuân theo khuôn dạng “yêu cầu – đáp ứng”.

I MỘT SỐ KHÁI NIỆM CƠ BẢN

- Kiến trúc của một hệ phần mềm có thể được thể hiện bởi **mô hình kiến trúc** (architectural model) bằng cách sử dụng những ký hiệu mô hình hóa.
- **Mô hình kiến trúc** là sản phẩm thể hiện một số hay tất cả những quyết định thiết kế tạo nên kiến trúc của hệ thống.
- Tập ký hiệu mô hình kiến trúc được gọi là **ngôn ngữ mô tả kiến trúc**: văn bản, đồ họa, biểu đồ...

| BÀI TẬP

- Một hệ thống thương mại điện tử như eBay, Amazon có cấu trúc gồm các thành phần, kết nối, cấu hình... Hãy khảo sát các hệ trên mạng và chỉ ra các yếu tố của các hệ thống này
- Hãy tìm hiểu các mô hình kiến trúc phần mềm phổ biến hiện nay.